Manual do aluno: Encontro 1, nuvem e versionamento

Ao final deste encontro, a sua dupla terá um site publicado automaticamente no GitHub Pages, com commits dos dois alunos e um pull request revisado. Siga os passos na ordem. Os trechos marcados como **Checkpoint** dizem como saber se você está no caminho certo. Vários passos pedem para você **prever** o resultado antes de executar: escreva a previsão no caderno e compare depois.

Se tiver dificuldade em qualquer passo, peça ajuda à sua IA favorita, como o Claude, o ChatGPT ou o Gemini. Diga em qual passo você está, cole a mensagem de erro inteira e conte o que você esperava ver. Peça que ela explique o motivo, e não só o comando. Antes de rodar qualquer coisa no seu repositório, leia a resposta e confira se faz sentido. Depois anote no caderno o que você aprendeu com a dúvida: o questionário pergunta o que você entendeu. A IA ajuda a destravar, mas o relatório e as respostas do questionário precisam ser seus.

Antes de começar

Faça isto antes de abrir qualquer ferramenta:

1. Crie a sua conta gratuita no GitHub (cada aluno, a sua) e confirme o e-mail. **Não cadastre cartão de crédito.**
2. Combine a dupla e decidam quem será o **aluno A** e quem será o **aluno B**. Anote o nome de usuário do GitHub de cada um.
3. Anote a **variação da dupla**, que o professor informa. Você usa esses três valores em todos os encontros.
4. Use um computador com navegador atualizado. Não precisa instalar nada: o Git, o Python e o Docker vêm no ambiente de trabalho, o Codespace, que você vai abrir no passo 3.

Papéis da dupla. O repositório é um só e fica na conta do aluno A. Nos passos, "(aluno A)" e "(aluno B)" dizem quem executa; "(os dois)" quer dizer que cada um faz no seu próprio computador.

Papel	O que faz
Aluno A	Cria o repositório, adiciona o B como colaborador, ativa o Pages, revisa e faz o merge dos pull requests
Aluno B	Aceita o convite, cria as branches e abre os pull requests
Os dois	Abrem, cada um, o seu Codespace no repositório do aluno A, fazem commits e `git push` com a própria conta

Variação da dupla. Anote no caderno os três valores que o professor passar:

Valor	O que é	Exemplo
`NOME_DUPLA`	Nome único da dupla	`dupla-07`
`MENSAGEM`	Frase que aparece no seu site	`Olá da dupla 07`
`PORTA`	Número da porta da sua API (você só usa a partir do encontro 2)	`5007`

Dois combinados valem para o projeto inteiro:

- O repositório da dupla é **público**. No plano gratuito do GitHub, o Pages só publica repositórios públicos.
- Cada aluno faz commits com a **própria conta**. O histórico de commits mostra quem fez o quê.

Se o seu GitHub estiver em português. Os nomes dos menus aparecem traduzidos. Alguns que você vai usar:

No manual	Pode aparecer como
Collaborators	Colaboradores
Codespaces	Espaços de código
Pages	Páginas
Source: GitHub Actions	Fonte: Ações do GitHub
Actions	Ações

Se um nome não bater, guie-se pela posição do item no menu e pelo ícone, ou pergunte ao professor.

1. do repositório ao site publicado

Situação: "Meu sistema precisa estar no ar sem o meu computador ligado, e a equipe não pode se atropelar nos arquivos."

Passo 1. Quadro IaaS, PaaS e SaaS. Depois da explicação do professor, preencha no caderno, para cada modelo: o que você gerencia, o que o provedor gerencia e um exemplo real que você já usou.

Passo 2. Criar o repositório (aluno A).

1. No GitHub, clique em **New repository** (Novo repositório). Nome: `projeto-devops-` seguido do seu `NOME_DUPLA`, por exemplo `projeto-devops-dupla-07`. Marque **Public** e **Add a README file**.
2. Em **Settings > Collaborators** (Configurações > Colaboradores), adicione o usuário do colega (aluno B). O aluno B aceita o convite pelo e-mail ou pela página do repositório.

Checkpoint: os dois enxergam o repositório, e o aluno B aparece como colaborador.

Passo 3. Abrir o Codespace (os dois). O aluno B só consegue depois de aceitar o convite. Cada um abre o **seu próprio** Codespace no repositório da dupla:

1. Na página principal do repositório, clique no botão verde **Code** (Código).
2. Abra a aba **Codespaces** (Espaços de código) e clique em **Create codespace on main** (Criar espaço de código na main).
3. Espere de 1 a 2 minutos. Abre um editor no navegador, com os arquivos à esquerda e um terminal embaixo. Não precisa digitar senha.

Quando o terminal aparecer, confira o estado do repositório:

```
git status
```

O resultado deve dizer `On branch main` e `nothing to commit`.

Checkpoint: o terminal está na pasta do repositório da dupla, e o `git status` não mostra erro.

Três cuidados com o Codespace:

- **Salvar o trabalho.** O que não foi enviado com `git push` só existe naquele Codespace, e Codespaces parados são apagados depois de 30 dias. Sempre termine o encontro com `git push`.
- **Parar.** Fechar a aba do navegador **não** para o Codespace. Ele para sozinho depois de cerca de 30 minutos sem uso. Para parar antes, abra o menu do Codespace (canto inferior esquerdo) e escolha a opção de parar o espaço de código.
- **Cota.** A conta gratuita tem uma cota mensal de horas de Codespace, e não precisa de cartão. Não deixe o Codespace aberto sem usar.

Conheça a tela do Codespace. Quando ele abre, você vê quatro áreas. Leia a tabela com a tela aberta na sua frente:

Área	Onde fica	Para que serve
Explorer (Explorador)	Barra da esquerda	Lista os arquivos e as pastas do repositório. É aqui que você cria um arquivo novo
Editor	No centro	Abre o arquivo em que você clicou, para escrever ou colar código. Salve com Ctrl+S

Área	Onde fica	Para que serve
Terminal	Embaixo	Onde você digita comandos como `git` e `python3`. Ele já começa na pasta do repositório
Aba PORTS (Portas)	Embaixo, ao lado da aba do terminal	Lista as portas abertas pelos seus programas, com um ícone para abrir a página no navegador

O menu de comandos do editor fica no ícone de três linhas, no canto superior esquerdo. Se algum item não estiver onde o manual diz, pergunte ao professor: o editor muda um pouco de aparência entre versões.

Voltar ao seu Codespace (encontros seguintes). Não crie um Codespace novo a cada encontro. Reabra o que você já tem:

1. Na página principal do repositório, clique no botão verde **Code** (Código) e abra a aba **Codespaces** (Espaços de código).
2. Na lista aparece o seu Codespace anterior, com um nome gerado pelo GitHub. Clique nele para reabrir. Costuma ser mais rápido do que criar um novo.
3. No terminal, rode `git pull` para trazer o que o seu colega enviou.
4. Se o seu Codespace não aparecer na lista, porque foi apagado ou você entrou com outra conta, crie outro como no passo 3. Os arquivos enviados com `git push` continuam no GitHub.

Um Codespace por aluno e por repositório basta. Criar vários gasta a sua cota de horas à toa.

Passo 4. Primeira página (aluno A). Preveja: o que aparece se eu abrir este arquivo no navegador? No painel **Explorer** (lista de arquivos, à esquerda), clique no ícone de **novo arquivo**, digite `site/index.html` e cole o código abaixo. Se não achar o ícone, clique com o botão direito numa área vazia do Explorer e escolha **New File** (Novo Arquivo).

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Projeto DevOps</title>
  <style>
    body { font-family: sans-serif; max-width: 40rem; margin: 2rem auto; background:
#ffffff; }
    h1 { color: #0b5cad; }
  </style>
</head>
<body>
  <h1>Projeto DevOps</h1>
  <p>Dupla: <strong>dupla-00</strong></p>
  <p>Mensagem: Olá, nuvem!</p>

  <h2>Sobre a dupla</h2>
  <ul>
    <li>Aluno A: <strong>SEU NOME</strong> (GitHub: SEU_USUARIO)</li>
  </ul>
  <p>Nossa frase: <em>escreva aqui uma frase da dupla</em></p>
</body>
</html>
```

Esta é a página **da sua dupla**, e vocês vão deixá-la com a cara de vocês. Antes de enviar, o aluno A personaliza a parte dele, no mesmo arquivo:

- Na seção **Sobre a dupla**, troque `SEU NOME` e `SEU_USUARIO` pelos seus dados.
- Combinem juntos uma frase da dupla e escrevam no lugar de `escreva aqui uma frase da dupla`.
- Mude a cor do título: em `h1 { color: #0b5cad; }`, troque o valor por uma cor de sua escolha, como `teal` ou `#c0392b`.

O aluno B acrescenta a linha dele e escolhe a cor de fundo no passo 6.

Para ver a página, suba um servidor simples dentro do Codespace:

```
python3 -m http.server 8000 --directory site
```

Quando aparecer o aviso da porta 8000, clique em **Open in Browser** (ou abra a aba **PORTS**, Portas, e clique no ícone de navegador dessa porta). Compare com a sua previsão e pare o servidor com Ctrl+C. Depois, envie ao GitHub:

```
git add site/index.html
git commit -m "Primeira página do site"
git push
```

Checkpoint: o arquivo `site/index.html` aparece no repositório, no GitHub.

Passo 5. Publicar com o GitHub Pages (aluno A).

1. No GitHub, vá em **Settings > Pages** (Configurações > Páginas). Em **Build and deployment** (Construção e implantação), escolha **Source: GitHub Actions** (Fonte: Ações do GitHub). Faça isso *antes* de enviar o fluxo do próximo item. Não clique em "Configurar" nos cartões sugeridos: o fluxo você cria à mão.
2. No Codespace, crie o arquivo `.github/workflows/pages.yml`. A pasta `.github` começa com ponto, e a indentação do YAML usa espaços, nunca tab. Se preferir o terminal, rode `mkdir -p .github/workflows` e depois `code .github/workflows/pages.yml`, que cria o arquivo e o abre no editor.

```
name: Publicar site

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  publicar:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deploy.outputs.page_url }}
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v4
        with:
          path: site
      - id: deploy
        uses: actions/deploy-pages@v4
```

3. Preveja: onde eu acompanho a publicação e quanto tempo ela leva? Depois, envie:

```
git add .github/workflows/pages.yml
git commit -m "Fluxo que publica o site no Pages"
git push
```

4. Abra a aba **Actions** (Ações) do repositório e acompanhe a execução até ficar verde. O endereço do site é `https://USUARIO.github.io/projeto-devops-NOME_DUPLA/`, com o seu usuário em minúsculas.

O que o fluxo diz, em três linhas:

- `on: push: branches: [main]` executa o fluxo a cada push na `main`.
- `permissions` autoriza o fluxo a publicar no Pages.
- `steps` baixa o código, prepara o Pages, empacota a pasta `site` e publica.

Checkpoint: o site abre no navegador e mostra `dupla-00`.

2: trabalhar em dupla com branch e pull request

Passo 6. Variação por branch e pull request. O aluno B faz a mudança, e o aluno A revisa. Preveja e escreva no caderno: o site muda quando eu abro o pull request ou quando faço o merge? Por quê?

1. Aluno B: no seu Codespace, atualize o repositório e crie uma branch.

```
git pull
git switch -c variacao-NOME_DUPLA
```

2. Aluno B: em `site/index.html`, troque `dupla-00` pelo seu `NOME_DUPLA` e `Olá, nuvem!` pela sua `MENSAGEM`. Na seção **Sobre a dupla**, acrescente a sua linha (copie a do aluno A, troque "Aluno A" por "Aluno B" e ponha o seu nome e o seu usuário) e mude a cor de fundo (`background: #ffffff;`) para uma cor de sua escolha, por exemplo `#eef4ff`.
3. Aluno B: envie a branch.

```
git add site/index.html
git commit -m "Aplica a variação da dupla"
git push -u origin variacao-NOME_DUPLA
```

4. Aluno B: no GitHub, clique em **Compare & pull request** e depois em **Create pull request**.
5. Aluno A: abra o pull request, vá na aba **Files changed**, confira a mudança e aprove em **Review changes > Approve**. Depois clique em **Merge pull request** e confirme.
6. Abra a aba **Actions** e veja quando o fluxo rodou. Compare com a sua previsão.

Checkpoint: o site mostra o nome e a mensagem da sua dupla, a seção Sobre a dupla com os dois alunos e as cores escolhidas. Se ainda mostrar a versão antiga, recarregue com Ctrl+F5.

Passo 7. Reunir as evidências (os dois). Atualize a `main` e veja o histórico:

```
git switch main
git pull
git log --oneline -5
```

Anote em um bloco de notas, porque o questionário do fim do encontro vai pedir:

- o código curto (7 caracteres) do commit mais recente da `main`, que é o merge publicado;
- o nome da branch;
- quem abriu o pull request e quem fez o merge;
- o nome do arquivo do fluxo e o que ele faz;
- o link do repositório;
- o link do site.

Passo 8. Fechar o quadro. Acrescente uma linha ao quadro do passo 1: em qual modelo (IaaS, PaaS ou SaaS) você colocaria o GitHub Pages? Justifique com a sua definição de quem gerencia o quê. Mais de uma resposta é defensável, e o que vale é a justificativa.

Erros comuns

Sintoma	Causa provável	O que fazer
O fluxo falha em `configure-pages`	O Source do Pages não está em GitHub Actions	Ajuste em Settings > Pages e use **Re-run all jobs** na aba Actions
`git push` recusado	Convite de colaborador não aceito, ou login na conta errada	Aceite o convite e entre com a sua própria conta
O site mostra 404	A primeira publicação ainda não terminou, ou `index.html` não está dentro de `site`	Espere o fluxo ficar verde e confira o caminho `site/index.html`
O site não mudou depois do merge	Cache do navegador, ou o fluxo falhou	Recarregue com Ctrl+F5 e abra o log na aba Actions
O Codespace não abre	O convite ainda não foi aceito (aluno B), ou a rede está bloqueando	Confirme o convite e avise o professor com a mensagem que apareceu
O Codespace parou sozinho	Ficou cerca de 30 minutos sem uso	Reabra em Code > Codespaces e envie o trabalho com `git push` logo em seguida

Relatório da dupla

Depois do passo 8, e antes do questionário, cada aluno escreve o que fez e o que aprendeu, e a dupla junta monta um relatório. Isso fixa o encontro e deixa claro quem fez o quê.

Parte 1: cada aluno escreve a sua parte

Aluno A e aluno B respondem **separadamente**, com as próprias palavras, em 3 a 6 linhas por pergunta. Não vale copiar do colega.

1. **O que eu fiz hoje.** Cite os arquivos que criei ou alterei, os comandos que rodei e o meu papel (aluno A ou B).
2. **O que eu aprendi.** Explique com as suas palavras o que é um repositório, um commit, uma branch e um pull request.
3. **Do `git push` até o site.** Descreva o caminho que o meu arquivo percorreu até a página aparecer no endereço do Pages.
4. **IaaS, PaaS ou SaaS.** Em qual modelo o GitHub Pages se encaixa, e por quê?
5. **O que deu errado ou foi difícil**, e como a dupla resolveu.
6. **Uma dúvida** que ainda tenho.

Mais uma pergunta, que depende do seu papel:

Papel	Pergunta extra
Aluno A	Por que foi preciso escolher **GitHub Actions** como fonte do Pages antes de enviar o arquivo do fluxo?
Aluno B	Por que a mudança foi feita em uma **branch**, com pull request, em vez de enviada direto para a `main`?

Parte 2: relatório da dupla

Um único relatório por dupla, escrito em conjunto, com estes itens:

- **Identificação:** o `NOME_DUPLA`, o nome e o usuário do GitHub de cada aluno, o link do repositório e o link do site.
- **Quadro IaaS, PaaS e SaaS** do passo 1, com a linha do GitHub Pages do passo 8.
- **Linha do tempo:** cole o resultado de `git log --oneline -5` e escreva, ao lado de cada commit, quem fez e o que mudou.
- **Um problema e a solução:** algo que travou a dupla e como foi resolvido.
- **O que a dupla aprendeu,** em três frases.

A entrega é feita pelo Moodle, como explica a seção Entrega, no fim deste manual.

Questionário

As perguntas dependem do repositório da sua dupla: commits, nomes de arquivos e os valores da sua variação. Por isso, **na hora de responder, deixe abertos o repositório, o site e a aba Actions**, ou vá anotando em um bloco de notas o que aconteceu, como pede o passo 7.

Antes de abrir o questionário, confira:

- ☐ O repositório da dupla está aberto em uma aba do navegador.
- ☐ O site publicado está aberto em outra aba.
- ☐ A aba **Actions** do repositório está aberta.
- ☐ As anotações do passo 7 estão à mão: o commit de 7 caracteres, o nome da branch, quem abriu o pull request, o nome do arquivo do fluxo, o link do repositório e o link do site.
- ☐ O quadro IaaS, PaaS e SaaS do caderno está à mão.

O questionário mistura três tipos de pergunta: conceitos (IaaS, PaaS e SaaS, Pages, pull request), evidências do **seu** repositório (links, commit, nomes de arquivos, a sua variação) e uma pergunta aberta, em que você explica com as suas palavras. Uma resposta com o dado certo do seu repositório vale mais do que uma resposta de memória.

Se o Codespace tiver parado, não tem problema: os commits, o histórico e a aba Actions continuam visíveis no GitHub, e você não precisa reabri-lo para responder.

Perguntas do questionário

O questionário tem seis blocos. Todos respondem os blocos 1, 2, 3 e 6. O bloco 4 é só do aluno A, e o bloco 5 é só do aluno B.

Bloco 1. Identificação (resposta curta)

- Nome completo.
- O `NOME_DUPLA`.
- O seu usuário no GitHub.
- O seu papel: aluno A ou aluno B.

Bloco 2. Conceitos (múltipla escolha, uma resposta certa)

1. Uma empresa aluga uma máquina virtual na nuvem e ela mesma instala o sistema operacional e o banco de dados. Que modelo é esse?

- a) IaaS
- b) PaaS
- c) SaaS
- d) Nenhum dos três

2. Você usa um editor de planilhas direto no navegador, sem instalar nada e sem cuidar de servidor. Que modelo é esse?

- a) IaaS
- b) PaaS
- c) SaaS

- d) Nenhum dos três

3. No modelo PaaS, quem gerencia o sistema operacional e o servidor onde o seu código roda?

- a) Você
- b) O provedor
- c) O usuário final
- d) Ninguém

4. Qual é a ordem correta para registrar e enviar uma mudança ao GitHub?

- a) `git push`, `git add`, `git commit`
- b) `git add`, `git commit`, `git push`
- c) `git commit`, `git push`, `git add`
- d) `git add`, `git push`, `git commit`

5. Para que serve um pull request?

- a) Baixar o repositório para o computador
- b) Propor uma mudança feita em uma branch e deixar um colega revisar antes de juntá-la à `main`
- c) Apagar uma branch antiga
- d) Publicar o site sem passar pela `main`

6. Neste projeto, o que faz o site ser atualizado depois de um push na `main`?

- a) O navegador do aluno B
- b) O Codespace, que fica ligado
- c) O fluxo do GitHub Actions definido em `pages.yml`
- d) Um comando manual do professor

Bloco 3. Evidências do seu repositório (resposta curta; consulte o repositório, o site, a aba Actions ou as suas anotações do passo 7)

7. Link do repositório da dupla.

8. Link do site publicado.

9. Código curto (7 caracteres) do commit mais recente da `main`. Dica: rode `git pull` e depois `git log --oneline -1`, ou olhe a lista de commits no GitHub.

10. Nome da branch que o aluno B criou para a variação.

11. Usuário do GitHub de quem abriu o pull request e de quem fez o merge.

12. Caminho completo do arquivo do fluxo dentro do repositório.

13. O `NOME_DUPLA` e a `MENSAGEM` que aparecem hoje no seu site, mais a frase da dupla da seção Sobre a dupla.

14. Na aba Actions, quanto tempo durou a última execução do fluxo **Publicar site**?

Bloco 4. Só para o aluno A (resposta curta)

15. O que você configurou em Settings > Pages antes de enviar o fluxo, e por que isso era necessário? Responda em 2 a 3 linhas.

16. Abra a aba Actions. Qual é o status da última execução (verde, vermelho ou em andamento) e qual commit a disparou?

Bloco 5. Só para o aluno B (resposta curta)

17. Por que a mudança foi feita em uma branch, com pull request, em vez de direto na `main`? Responda em 2 a 3 linhas.

18. Na aba **Files changed** do seu pull request, quais arquivos foram alterados e o que mudou neles?

Bloco 6. Perguntas abertas (todos; responda com as suas palavras)

19. Descreva o que acontece desde o `git push` até o site mudar no ar.

20. Em qual modelo (IaaS, PaaS ou SaaS) você colocaria o GitHub Pages? Justifique dizendo quem gerencia o quê.

Entrega

Como fazer o relatório.

1. Cada aluno escreve a **Parte 1**, sozinho, com as próprias palavras.
2. Juntos, vocês montam a **Parte 2**, o relatório da dupla: identificação, quadro IaaS, PaaS e SaaS, linha do tempo dos commits, um problema resolvido e o que a dupla aprendeu.
3. Antes de fechar, confiram se o link do repositório e o link do site abrem, e se os commits dos dois alunos aparecem na linha do tempo.
4. Juntem tudo em um único arquivo (por exemplo, em PDF), com o `NOME_DUPLA` no nome do arquivo.

Entrega no Moodle. No final da tarefa, a dupla entrega o relatório pelo Moodle. O relatório é o mesmo para os dois, mas **cada aluno entrega o seu, separadamente, pela própria conta**. Confira se o envio foi concluído antes de sair.